CRC Package Interface Specification

Preparation, model-matrix, initialization, E-step, capture-model M-step, outcome likelihood, outcome-model M-step, and public EM fitting interface

1 Preparation interface

The internal function `prepare_crc()` now owns the complete pre-fit pipeline. It resolves the outcome distribution and control settings, validates the public inputs, parses capture indicators, outcomes, formulas, and misclassification specifications, performs cross-component compatibility checks, constructs the model matrices, and initializes the state weights and capture-cell counts. It returns an object of class `crcfit_preparation` containing the prepared model components and the validated fitting settings.

`crc_fit()` records the matched call and delegates preparation to `prepare_crc()`. It transfers the prepared components into its local environment and calls `perform_em_crc()` to fit the model. The local variables created through `list2env()` are declared explicitly so that static R code analysis can identify their bindings. This separation allows unit tests to exercise preparation and the internal fitting components independently while the public function provides the complete fitting path.

2 Model-matrix infrastructure

The package now converts parsed model specifications into complete capture-cell grids and aligned observation-level states. The infrastructure includes the capture and outcome design matrices, misclassification contributions, and a direct mapping from each observation state to its capture cell. Construction of these components is integrated into `prepare_crc()`, which is called by `crc_fit()`. The capture- and outcome-model M-steps are coordinated by the internal EM algorithm, which is now integrated into `crc_fit()`.

2.1 Capture-model matrix

`build_capture_matrix()` constructs the Cartesian product of all capture histories, categorical levels used by the capture model, and latent classes. For misclassified variables, true-category levels and their ordering are obtained from the corresponding confusion matrices. The function returns:

- `cells`, the complete capture-model cell grid;
- `matrix`, the design matrix generated from the parsed terms;
- `unobserved`, an indicator of all-zero capture histories.

With J capture sources, K latent classes, and categorical variables with $C_1, \ldots, C_p$ levels, the grid contains $2^J K \prod_{q=1}^{p} C_q$ rows.

2.2 Initial observation states

`build_observation_states()` repeats every observed unit once for each latent class. It retains the variables needed by the capture model, outcome model, and misclassification mechanisms. It also keeps aligned copies of the standardized outcome and observed values of misclassified variables.

An observation identifier maps every expanded row back to its original unit. Only variables used by the capture or outcome model are retained in the state data. Recorded values of variables subject to misclassification are stored separately for evaluation of their confusion-matrix contributions.

2.3 Misclassified-variable expansion

`expand_misclass_states()` expands the current states over every possible true level of one misclassified variable. Each expanded row receives the contribution

$$\Pr(\widetilde{G}_j = h \mid G_j = g)$$

from its confusion matrix. When `true_if` applies, the observed category receives probability one and all other candidate categories receive probability zero.

For several misclassified variables, separate confusion matrices are used and their contributions are multiplied. This corresponds to conditional independence of the misclassification mechanisms given their respective true categories. Dependent mechanisms may be supported in a future extension. These contributions are likelihood terms, not posterior probabilities, and must not be normalized at this stage.

`expand_all_misclass_states()` applies this expansion successively to every specified variable. When no misclassification mechanism is supplied, it assigns a contribution of one to every latent state.

2.4 Outcome-model matrix

`build_outcome_matrix()` applies the parsed outcome terms to the fully expanded state data. Its rows therefore remain aligned with the state data, standardized outcomes, observation identifiers, and accumulated misclassification contributions. It returns `NULL` when no outcome model is specified.

Every predictor in `outcome_formula` is required to occur in `capture_formula`. This ensures that the predictor is represented in the complete capture-cell grid and is consequently available for prediction in the all-zero cells. The formulas may contain different terms; only the set of variables is constrained.

2.5 Capture-cell indices

`build_capture_index()` returns one integer index for every expanded observation state. Matching uses the capture indicators, categorical variables in the capture model, and latent class. If the ith value is r, row i of the state data uses row r of the capture-cell grid and capture design matrix. The vector avoids repeated joins during EM iterations. Construction fails if any state cannot be matched to the grid.

2.6 Unified model-matrix object

`build_model_matrices()` coordinates the complete construction sequence and returns an object of class `crc_model_matrices` with:

- `capture`, containing the cell grid, capture design matrix, and all-zero-cell indicator;

- `states`, containing expanded state data and aligned metadata;
- `outcome_matrix`, containing the outcome design matrix or `NULL`;
- `capture_index`, linking state rows to capture-cell rows.

After parsing and compatibility validation, the preparation object stores this component under the name `model_matrices`. The value is retained by `crc_fit()` for interpretation and subsequent prediction.

2.7 Cross-component validation

Compatibility checks now reject outcome predictors absent from the capture model. They also reject variables specified in `misclass` that are used by neither the capture nor the outcome model. This prevents redundant state expansion and makes the population-cell interpretation explicit.

3 Initialization

`initialize_crc()` creates normalized initial weights for the expanded observation states and expected counts for the complete capture-cell grid. For multiple latent classes, independent gamma draws produce a symmetric Dirichlet allocation within each observation and candidate true-category configuration. These allocations are multiplied by the misclassification contributions and normalized within each observed unit. The concentration is set by `control$init_alpha` and defaults to 20.

State weights are aggregated through the precomputed capture-cell indices. Observed cells receive the corresponding weighted counts, while every all-zero capture cell receives an initial expected count of one. The resulting `crc_initialization` object stores the state weights, cell counts, and Dirichlet concentration. It is returned by `prepare_crc()` and retained by `crc_fit()`.

4 Control and progress interface

`parse_control()` resolves the user-supplied `control` list against complete defaults and returns an object of class `crc_control`. The resolved object is stored by `prepare_crc()` and subsequently by `crc_fit()`. Partial overrides are supported, while unknown, duplicated, unnamed, and invalid settings are rejected. The available settings are:

- `init_alpha`, the positive Dirichlet concentration, with default 20;
- `em$max_iter` and `em$tolerance`, with defaults 1000 and 10^{-6};
- `capture$max_iter` and `capture$tolerance`, with defaults 100 and 10^{-8};
- `outcome$max_iter` and `outcome$relative_tolerance`, with defaults 1000 and 10^{-8}.

The separate logical argument `verbose` defaults to `FALSE`. It is validated and stored by `crc_fit()` and accepted by the internal EM driver. User-facing iteration messages remain to be implemented; optimizer-specific tracing is not part of the public control interface.

5 Expectation step

`expectation_crc()` implements the distribution-independent part of the E-step. It accepts one fitted capture-model mean for every row of the complete capture-cell grid and, when an outcome model is present, one outcome log-likelihood contribution for every expanded observation state. A missing outcome model is represented by zero log-likelihood contributions, equivalent to likelihood contributions of one.

For state (i, g, k), the unnormalized log weight is

$$\log m_{Y_i,g,k} + \log \Pr(\widetilde{G}_i \mid G_i = g) + \log L_V(i \mid g).$$

Exact zero misclassification probabilities remain negative infinity on the log scale and therefore produce posterior weights of exactly zero. The function verifies that each observed unit has at least one admissible state and normalizes the weights within unit after subtracting the largest log weight. This log-sum-exp calculation avoids unnecessary underflow and overflow.

The normalized state weights are aggregated through `capture_index` to update the expected counts for observed capture histories. Expected counts for all-zero histories are set to their current fitted capture-model means. The function returns a `crc_estep` object containing one posterior weight

per expanded state and one expected count per capture cell. It is called by `em_iteration_crc()` during every internal EM iteration, including iterations initiated through the public fitting function.

6 Capture-model maximization

`maximize_capture_crc()` implements the capture component of the M-step. Given expected capture-cell counts n_c and capture-model rows x_c, it maximizes

$$Q_Y(\theta) = \sum_c \{n_c \log m_c(\theta) - m_c(\theta)\}, \qquad m_c(\theta) = \exp(x_c^\top \theta).$$

The maximization is performed with `glm.fit()` using a Poisson family and logarithmic link. Fractional expected cell counts are valid for this objective because the omitted terms do not depend on θ. The specific Poisson warning about non-integer counts is suppressed, while other warnings remain visible.

When no starting values are supplied, all coefficients start at zero, giving initial fitted cell means of one. The function also accepts previous coefficients as warm starts for later EM iterations. The public capture controls are translated to `glm.control()` by `em_iteration_crc()`.

The function rejects invalid counts or starting values, non-convergence, rank-deficient design matrices, non-finite coefficients, and non-positive or non-finite fitted means. It returns a `crc_capture_mstep` object with the coefficients, fitted cell means, maximized objective value, iteration count, and convergence indicator.

7 Outcome log-likelihood

`outcome_loglik_crc()` evaluates one outcome log-likelihood contribution for every expanded observation state. Its arguments are the aligned standardized outcome table, a positive mean `mu` for every state, the outcome distribution, and, for the negative binomial model, a single positive dispersion parameter `sigma`. Predictors in `outcome_formula`, including `.latent`, may therefore produce different means for different candidate states of the same observed unit.

Let W denote the corresponding untruncated count. Under `"ztpois"`, $W \sim \text{Poisson}(\mu)$. Under `"ztnegbin"`, W has mean μ, size $1/\sigma$, and variance $\mu + \sigma\mu^2$. In both cases the modeled outcome is the conditional variable $V = W \mid W \geq 1$. If f, F, and $S(q) = \Pr(W > q)$ denote the untruncated mass, distribution, and survival functions, respectively, the implemented contribution is

$$\ell_V = \begin{cases} \log f(v) - \log S(0), & \text{exact observation,} \\ \log\{S(l-1) - S(u)\} - \log S(0), & \text{interval } l \leq V \leq u, \\ \log S(l-1) - \log S(0), & \text{right censoring } V \geq l, \\ 0, & \text{missing outcome.} \end{cases}$$

Thus, interval bounds are inclusive and missing outcomes contribute a likelihood factor of one under the ignorable coarsening assumption. Exact contributions use the log probability mass directly. Interval contributions use upper-tail log probabilities and `log_sub_crc()`, avoiding unstable subtraction of nearly equal ordinary probabilities in the right tail. No artificial probability floor is applied.

7.1 Log-scale probability subtraction

`log_sub_crc()` provides the stable operation

$$\log\{\exp(a) - \exp(b)\}, \qquad a \geq b,$$

using `expm1()`. This helper is used for interval-censored outcome likelihoods, where the required probability is obtained by subtracting two survival probabilities. It also handles the case in which both probabilities are zero and their log values are negative infinity.

8 Outcome-model maximization

The internal function `maximize_outcome_crc()` implements the outcome component of the M-step by maximizing the posterior-weighted expected log-likelihood

$$Q_V(\phi) = \sum_s \tau_s \ell_V(s; \phi),$$

where s indexes the expanded observation states, τ_s is the current posterior state weight, and $\ell_V(s;\phi)$ is provided by `outcome_loglik_crc()`. Missing outcomes have zero log-likelihood contributions and therefore do not affect this maximization.

The function minimizes $-Q_V$ with `optim()` using `method = "BFGS"`. For `"ztpois"`, the parameter vector contains only the outcome regression coefficients β, with $\mu_s = \exp(z_s^\mathsf{T}\beta)$. For `"ztnegbin"`, it also contains `log_sigma`; optimizing the logarithm guarantees $\sigma > 0$ without box constraints. No arbitrary coefficient bounds or user-selected optimizer are exposed. The implementation currently uses numerical derivatives.

Preliminary regression coefficients are obtained from a posterior-weighted Poisson fit to exact outcomes when there are enough distinct exact observations and the preliminary design matrix has full rank. Otherwise, the intercept is initialized as the logarithm of the weighted mean of informative lower bounds and the remaining coefficients are set to zero. For the negative binomial model, a positive method-of-moments dispersion estimate based on exact outcomes is used when at least two distinct exact observations are available. The estimate is bounded below by 10^{-4}, while 0.25 is the conservative fallback. These quantities are starting values only and do not replace maximization of the zero-truncated, censored objective.

The function accepts the preceding outcome coefficients and `log_sigma`, when applicable, as warm starts. Named starting vectors are validated and reordered to match the outcome design matrix. The `max_iter` and `relative_tolerance` elements of `control$outcome` are translated to the corresponding `optim()` controls by `em_iteration_crc()`.

Only positive-weight states enter the weighted objective, which avoids undefined products of zero weights and negative-infinite log likelihoods. Invalid likelihood evaluations receive a finite penalty. An update is accepted only when `optim()` reports convergence code zero, the objective is finite, and Q_V has not decreased beyond a small numerical tolerance. A failed warm-start optimization is retried from the conservative default values. The function also rejects rank-deficient informative design matrices and non-finite or non-positive fitted means or dispersion values.

The returned `crc_outcome_mstep` object contains the regression coefficients, fitted state means, dispersion for the negative binomial model, maximized objective, optimizer evaluation counts, retry indicator, and convergence indicator.

9 EM algorithm

9.1 Single iteration

`em_iteration_crc()` coordinates one complete EM update. It first fits the capture model from the current expected capture-cell counts. When an outcome model is present, it then fits that model using the current state weights and evaluates the resulting outcome log-likelihood for every expanded state. Finally, it calls `expectation_crc()` with the fitted capture means and outcome contributions. Models without an outcome skip the outcome maximization and use zero log-likelihood contributions in the E-step.

The function translates the resolved capture and outcome controls to `glm.control()` and `optim()` controls, respectively. It accepts optional starting values for both M-steps. Its `crc_em_iteration` result contains the `capture_fit`, optional `outcome_fit`, updated `state_weights`, and updated `cell_counts`.

9.2 Iteration loop and warm starts

The loop starts from the `crc_initialization` object. `perform_em_crc()` repeatedly applies the single-iteration update. After the first iteration, the preceding capture coefficients are used as the capture-model starting values. The outcome coefficients are likewise reused; for `"ztnegbin"`, the preceding dispersion is appended as `log_sigma`. The loop stops at convergence or after `control$em$max_iter` iterations.

The returned `crc_em_fit` object contains the final component fits, state weights, expected cell counts, number of iterations, convergence indicator, and convergence diagnostics. Reaching the iteration limit returns the final update with `converged = FALSE`.

9.3 Public fitted object

`crc_fit()` stores the EM result as `em_fit`, retains the validated specifications, model matrices, initialization, call, and input data, sets `fitted = TRUE`, and returns an object of class `crc_fit`. If the EM iteration limit is exhausted, the final result is returned with its non-convergence indicator and a warning is issued. This keeps the last estimates available for diagnostic inspection.

9.4 Convergence criterion

For iteration t, the loop calculates

$$\begin{aligned}
\delta_{\tau}^{(t)} &= \max_{s} |\tau_s^{(t)} - \tau_s^{(t-1)}|, \\
\delta_{Y}^{(t)} &= \max_{c} |\log m_c^{(t)} - \log m_c^{(t-1)}|, \\
\delta_{V}^{(t)} &= \max_{s} |\log \mu_s^{(t)} - \log \mu_s^{(t-1)}|, \\
\delta_{\sigma}^{(t)} &= |\log \sigma^{(t)} - \log \sigma^{(t-1)}|.
\end{aligned}$$

Here s indexes expanded observation states, c indexes capture cells, τ denotes posterior state weights, m denotes fitted capture means, and μ denotes fitted outcome means. The outcome-mean

term is omitted when no outcome model is present, and the dispersion term is included only for `"ztnegbin"`. Explicitly monitoring dispersion makes the package criterion stricter than the simulation implementation, in which dispersion was not a separate convergence component.

The overall change is the maximum of the applicable components. Convergence requires this value to be finite and strictly smaller than `control$em$tolerance`. The first iteration has infinite change because no preceding fitted values exist, so at least two iterations are required. Log differences provide scale-free, approximately relative changes for positive model quantities; state probabilities are compared directly because zero is admissible.

10 Automated tests

The public fitting interface, model-matrix, initialization, control, E-step, both M-steps, outcome-likelihood, and EM components are covered by automated tests implemented with the `tinytest` package. Separate test files cover model matrices, misclassification expansion, cross-component compatibility, initialization, control parsing, expectation calculations, maximization, outcome log-likelihood calculations, EM orchestration, and public fitting.

The existing tests cover capture grids and design matrices, row alignment, factor levels, all-zero cells, misclassification expansion, compatibility, initialization, and E-step behavior. Test fixtures for these components now call `prepare_crc()` directly when testing parsed inputs, model matrices, or initialization objects. The E-step and both M-steps remain tested directly, while the dedicated EM tests exercise their internal orchestration.

Initialization tests verify normalized, reproducible state weights and their capture-cell aggregation, including the single-class case and invalid control values. Fifteen E-step assertions cover posterior normalization, impossible states, cell aggregation, numerical stability, dimensions, and validation.

Thirteen control assertions cover defaults, partial nested overrides, the `verbose` argument, unknown settings, malformed subsections, and invalid values. Twelve capture-maximization assertions cover fitted-object structure, dimensions, finite coefficients and means, objective evaluation, warm starts, invalid inputs, and rank deficiency.

Fourteen outcome-maximization assertions cover the Poisson and negative binomial models, coefficient names, positive fitted means and dispersion, finite objectives, reordered named warm starts, retry from an invalid warm start, invalid state-weight dimensions, and rank-deficient informative design matrices. Together, the capture- and outcome-maximization test file contains 26 assertions.

The outcome-likelihood tests compare the Poisson and negative binomial results with direct probability calculations for mixed exact, interval-censored, right-censored, and missing outcomes. They also verify inclusive interval bounds, a right-censoring boundary at one, numerical stability in the far right tail, and validation of the mean and dispersion inputs. The complete outcome-likelihood test file contains nine assertions.

The dedicated `test_em.R` file contains 23 assertions. It verifies single-iteration result classes, finite fitted values, positive negative binomial dispersion, normalized state weights, and accepted warm starts. It also exercises full-loop convergence for negative binomial and Poisson outcomes, operation without an outcome model, the convergence-component structure, and termination with `converged = FALSE` after a one- iteration limit.

The dedicated `test_crc_fit.R` file adds 14 end-to-end assertions. It covers a converged zero-truncated Poisson outcome model, a model without an outcome component, the fitted object and

internal component classes, retained call and distribution information, and the warning and returned diagnostics when the EM iteration limit is exhausted. The complete suite now contains 142 passing assertions.

11 Planned next steps

The core fitting path and its component-level and end-to-end tests are now in place. Development should continue with the following main steps.

1. Implement and test a `predict()` method for overall population, categorical-subpopulation, and latent-class estimates.
2. Validate fitting and prediction using the accompanying IAOS submission data and simulation setting.
3. Add user-facing methods such as `print()` and `summary()`.
4. Extend the package documentation, `NEWS.md`, examples, and vignettes, and implement the remaining progress reporting.

EM updates must follow the methodology in the accompanying paper and simulation implementation. The existing automated tests establish the initialization, matrix, alignment, iteration, and convergence invariants on which the public fitting interface will depend.